

年轻人的第一个爱马仕 可以是个智能体

从爱马仕开始的传奇 | From Harness to Agent

Nous Research 出品 · 华子 分享

爱马仕：189 年的传奇

1837 年，Thierry Hermès 在巴黎开设马具工坊
那时马是主要交通工具，贵族出行全靠马车
他观察到：市面上马具质量参差不齐
很多马因不合身的挽具而动作受限

他的解决方案很朴素：做最好的 Harness



「我们的第一个客户是马」
Harness 的设计要跟被约束者对齐 (alignment)
—— 爱马仕早期产品哲学，流传了六代人



爱马仕的核心哲学

对齐 Alignment

「第一个客户是马」
与被约束者对齐
好的设计让被约束者
感觉不到约束的存在

容错 Fault Tolerance

鞍针缝法 (saddle stitch)
两根针、两根蜡线
从相反方向穿过皮革
一根断了另一根撑住

品控 Quality Control

识别「幻觉」
外观精致但结构有隐患
三层护栏：
工匠自检→团队复查→签字

克制 Restraint

「为剔除而构建」
砍掉 80% 产品线
效率反而上去了
变的是承载物，不变的是方法论

这四条哲学，在 189 年后被一个 AI Agent 项目完整继承



从马具到智能体

爱马仕品牌

- 1837 年，马具工坊起步
- 从马具 → 皮具 → 丝巾 → 香水
- 6 代传承，核心是「工艺 + 进化」
- 2024 年营收 152 亿欧元
- 全球最成功的 Harness 公司

Hermes Agent

- 2025 年，Nous Research 开源
- 从工具集 → 技能 → 自进化智能体
- 核心理念：「经验 + 学习 + 进化」
- 一个月 5 万 GitHub stars
- 第一个 inference-time learning agent

同样的精神：**Harness** 约束引导力量方向，**Agent** 约束引导 LLM 能力释放
马的时代结束了，马具的哲学还在



Hermes Agent: 自进化个人智能体

Nous Research 出品 — 第一个把「运行时持续学习」做成主线产品的开源框架

- ▶ 不是 Web 应用，不是 SaaS 平台
- ▶ 是一个运行在终端的 Python CLI 程序
- ▶ 9431 行核心编排器 + 数十个工具
- ▶ 18 个 LLM provider 随便切
- ▶ 17 个消息平台原生接入
- ▶ pip install 即运行，5 美元 VPS 即可

GitHub: [nousresearch/hermes-agent](https://github.com/nousresearch/hermes-agent)

维度	Claude Code	OpenClaw	Hermes Agent
诞生母体	Anthropic 产品团队	开源社区 + 大厂赞助	Nous Research 训练实验室
抽象层	IDE/终端工具	多渠道平台	单用户长期伙伴
License	Proprietary	MIT	MIT, 零 telemetry
主要交互入口	终端 / IDE 插件	23+ 消息渠道 + Web + macOS/iOS/Android	CLI + 6 个消息渠道 + 邮件 + 语音
会话记忆	自动 memory (近期才加)	数据库存所有, token 成本随时间增长	容量受限的 MEMORY.md (2,200 字符) + FTSS 历史检索
用户建模	弱	中 (持久化对话)	强 (Honcho 辩证用户模型)
Skill 来源	基座模型能力 + 手写 skills	社区市场 ClawHub (13k+)	agent 自创建 + 跨框架 agentskills.io ^[1] 标准
Skill 的"学习"在哪发生	训练时 (Anthropic 那边)	人类社区贡献	inference time, 用户自己的轨迹
模型支持	锁定 Anthropic	BYOK: OpenAI / Anthropic / Google / DeepSeek / 本地	18+ provider, 含 Nous Portal / OpenRouter / Kimi / MiniMax / 通义 / MiMo / 本地
核心逻辑	大模型	本地或云端/边缘部署	本地 / Docker / K8S / Kubernetes / ...
核心逻辑	anthropic		
核心逻辑	112k / ... ode ^[12]		
核心逻辑	桌前的 pair-programmer	装满工具的瑞士军刀	跟你一起长出来的实习生

5

入口层

9400+

核心编排

17

消息平台

26

技能类别

400+

测试用例

四个「设计特点」

那些在项目早期做出的、难以逆转的、影响全局的架构选择

0
1

Learning Loop

从经验中学习

Agent 能从工作中提炼技能
每 10 次工具调用自动检查
后台 AI 审查对话历史
使用时发现过时立即 patch

0
2

CLI-First

终端是第一公民

cli.py 8736 行完整 TUI
prompt_toolkit 多行编辑
流式输出 + slash 命令
六种终端后端

0
3

Personal

个人的、长期的

SessionDB 跨会话持久化
MEMORY.md + USER.md
FTS5 全文搜索历史
Honcho 用户建模

0
4

Run Anywhere

从 \$5 VPS 到 GPU

零外部依赖
SQLite 单文件部署
pip install 即运行
512MB RAM 即可工作

「一个 agent 如果不能从经验中学习，就只是一个更好的 chatbot」

五层架构设计

入口层

CLI / TUI / Gateway / Batch / Cron

5 种触发方式，共享同一 Agent 内核

编排层

run_agent.py (9431行) / model_tools.py

对话循环、模型调用、工具调度

能力层

tools/ (数十个) / skills/ (26类) / plugins/

工具注册、技能管理、插件系统

状态层

hermes_state.py (SQLite) / memory_manager.py

会话持久化、记忆编排、上下文压缩

平台层

gateway/platforms/ (17个) / acp_adapter/

Telegram / Discord / Slack / 微信 / 飞书 / 钉钉...

▲ 上层依赖下层，下层不感知上层 | 单向依赖

AI Agent 核心引擎

核心循环: 构建消息 → 调用模型 → 解析工具调用 → 执行工具 → 循环

迭代控制

Iteration budget 管理
防止 Agent 无限循环

智能降级

Fallback provider 切换
主模型不可用自动降级

流式输出

11 个回调接口
适配 CLI / Gateway / Batch

成本追踪

Token 使用量统计
实时成本估算

记忆编排

多提供者记忆管理
prefetch / sync 模式

上下文压缩

LLM-based 摘要压缩
token budget 尾部保护

工具 & 技能生态

工具系统 (Tools)

- Terminal — 6 种后端
- File Tools — 文件读写搜索
- Browser — Playwright 自动化
- Delegate — 子代理委托
- MCP — Model Context Protocol
- Code Execution — Python/JS
- Session Search — 跨会话检索
- Process — 后台进程管理

技能系统 (Skills)

- 26 个技能类别
- Apple/Mac 自动化
- GitHub 工作流
- MLOps 全链路
- Research & 写作
- Creative 创意生成
- DevOps 运维
- 自主创建 + 自动改进

工具 = 「能做什么」 技能 = 「怎么做」 插件 = 「扩展能力」

LLM Wiki: 知识复利引擎

Karpathy 的 LLM Wiki 已内置为 Hermes 技能 — 一条命令启动知识编译

三层架构

第一层: **raw** (原始来源)

你的原始材料 — LLM 只读不改

第二层: **wiki** (编译层)

LLM 编译出的页面网络 [[wikilink]]

第三层: **SCHEMA** (规则层)

你和 AI 的契约 — 规则本身还能进化

传统 RAG vs LLM Wiki

传统 RAG: 每次现捞

像每次饿了才下厨, 材料都是生的

LLM Wiki: 先编译再用

一篇文章可能触发十几个页面更新

Obsidian 是 IDE, LLM 是程序员, Wiki 是代码库

实操: ① /llm-wiki 初始化 → ② 投喂材料 → ③ 提问触发编译 → ④ 定期 lint 体检

全平台消息网关

Gateway 同时连接多个平台，每个平台独立会话管理

Telegram

Discord

Slack

WhatsApp

Signal

Matrix

Email

飞书

企业微信

个人微信

钉钉

Mattermost

Home Assistant

SMS

REST API

Webhook

17 个平台类型 | Webhook + Polling | 会话隔离 | WAL 并发安全

Self-Improving: 自进化能力

1

发现 Discover

扫描使用模式
识别重复操作

2

提取 Extract

从交互中提取
结构化知识

3

生成 Generate

基于模板自动
创建新技能

4

改进 Improve

分析失败模式
迭代优化技能

▸ Inference-time Learning: 运行时即训练时

每次 5 步以上的 tool 序列、每次 self-correction、每次用户反馈
都被当成一条值得提炼的「高质量 trajectory」

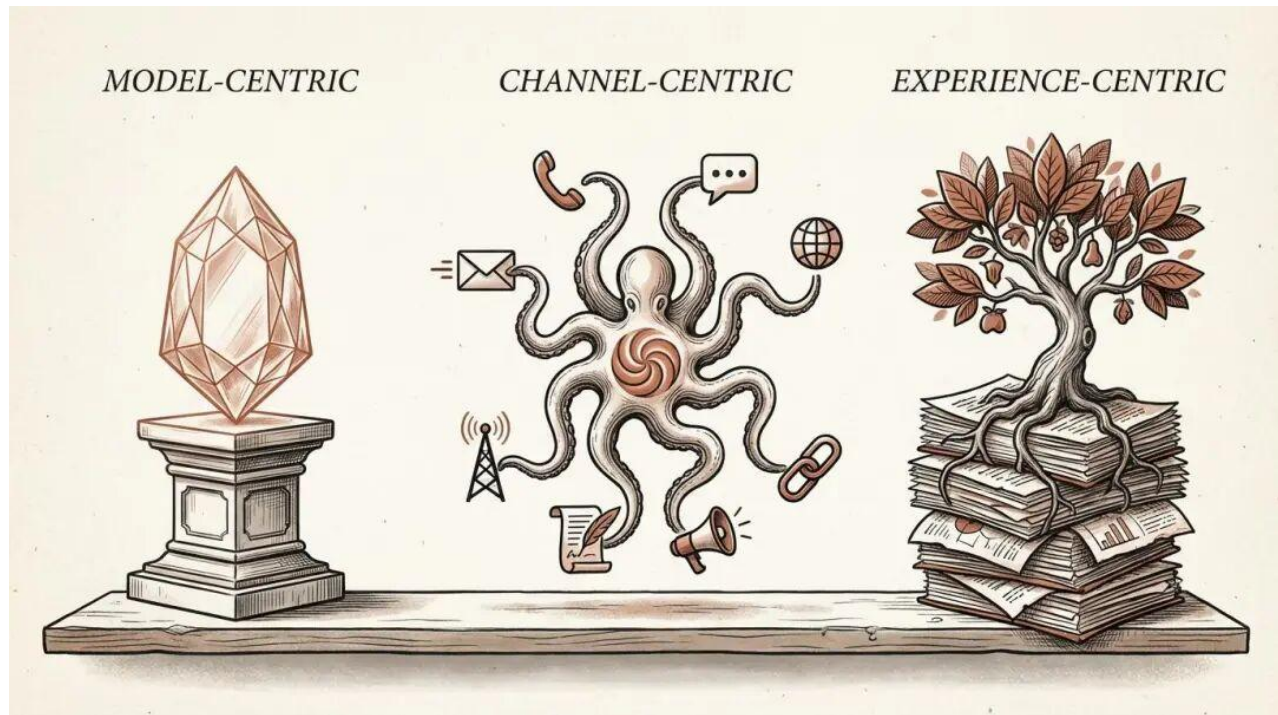
▸ Memory Nudge: 每 10 次工具调用自动检查是否应该创建技能

▸ Background Review: 后台线程启动独立 AI Agent 审查对话历史

▸ 使用时改进: 发现技能过时立即 patch, 形成「使用→反馈→改进」正反馈循环

三种 Agent 哲学

Claude Code · OpenClaw · Hermes Agent — 三种完全不同的设计假设



Claude Code

- 单模型绑定 Claude
- SWE-bench 80.8%
- 编码专用，短会话模型

OpenClaw

- 22+ 消息渠道
- ClawHub 13,000+ skills
- 飞书生态深度整合

Hermes Agent

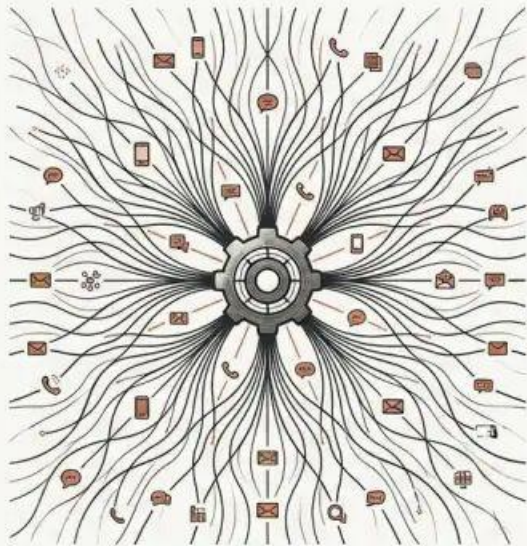
- 多模型 + 持久学习
- inference-time learning
- 通用工具 + 全平台 Gateway

过去两年只在两个维度扩 agent: 模型能力(X) × 渠道广度(Y)。Hermes 启动了第三个维度: 经验积累(Z)

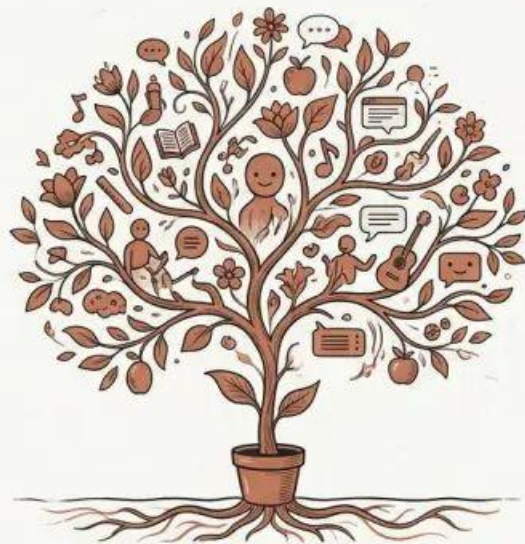
与同类工具的差异



MODEL



CHANNELS



EXPERIENCE

THE EVOLUTION OF AI AGENTS

Claude Code (模型中心论)：能力上限 $\approx$ 基座模型，赌「更强模型 + 最薄 harness」

OpenClaw (渠道中心论)：价值在于「无处不在」，赌「更广渠道 + 更大市场」

Hermes Agent (经验中心论)：价值在于「长期共生」，赌「运行时学习 + 长期沉淀」

Hermes vs OpenClaw 深度对比

同时跑了三个月的真实体验，不是云评测

记忆系统

Hermes: 三层记忆网络

会话/持久/技能记忆 — 灵活性强，跨平台无缝衔接

OpenClaw: L1/L2/L3 + EvoMap

记忆蒸发有节奏，不污染上下文 — 稳定性更强

技能生态

Hermes: 40+ 内置 + 自创建

多平台覆盖，无 Lock-in，18 个 provider 随便切

OpenClaw: ClawHub 13,000+ 插件

飞书原生深度整合（日历/文档/多维表格/任务）

核心结论

个人用户/极客 → **Hermes:** 多平台 + 灵活

中国企业/飞书 → **OpenClaw:** 原生整合 + 稳定

IDE 程序员 → **Claude Code:** SWE-bench 80.8%

Hermes 路线上的五个开放问题

NousResearch/ **atropos**



Atropos is a Language Model Reinforcement Learning Environments framework for collecting and evaluating LLM trajectories through diverse environments

 110

Contributors

 18

Used by

 1k

Stars

 322

Forks



Hermes Agent

不只是一个 AI 助手，而是一个会成长的智能伙伴

5

入口层

9400+

核心编排

17

消息平台

26

技能类别

400+

测试用例

GitHub: [nousresearch/hermes-agent](https://github.com/nousresearch/hermes-agent)

开源 · 可扩展 · 自进化

从 Harness 到智能体 — 与爱马仕同名，共享「约束引导、自进化、知识复利」的精神